

Tier 2 Stack Deep-Dive: Neon + Render + Cloudflare Pages для FastAPI/Postgres/Vite (2026)

TL;DR

DNS-проблема — это не строка подключения Neon, а известный баг сети WSL2/Docker Desktop: запуск Docker Desktop переписывает/ломает `/etc/resolv.conf` внутри WSL2, из-за чего контейнер не резолвит внешние хосты (`Temporary failure in name resolution`), хотя Compose-сеть между сервисами по DNS-именам продолжает работать через встроенный резолвер Docker `127.0.0.11`. Надёжный фикс: запускать миграции **не из контейнера**, а прямо из WSL2 (`psql "$NEON_DIRECT_URL" -f migration.sql`), либо явно прописать DNS в Docker Engine (`daemon.json`) и/или закрепить `resolv.conf` в WSL2.

Единого «золотого пути» из коробки для FastAPI+Neon+Render+Cloudflare Pages нет: только отдельные официальные point-to-point гайды (Neon→Render — но под Node, Neon→Cloudflare Pages, Render→Cloudflare как DNS). Ключевые «острые углы»: transaction-pooling Neon (нет `SET`/`LISTEN`/`SQL-PREPARE`), scale-to-zero убивает idle-соединения вашего пула, Render free spin-down 15 мин + Python 3.14.3 по умолчанию.

CORS для Bearer-токена тривиален: при чисто header-based авторизации (без cookies, без `credentials:'include'`) `allow_credentials` НЕ нужен — ставьте `allow_credentials=False`, точный список `allow_origins`, `Authorization` в `allow_headers`; preflight OPTIONS Starlette обрабатывает автоматически.

Key Findings

DNS изнутри контейнера ломается из-за WSL2-networking, а не Neon. Docker/Microsoft issue-трекеры прямо фиксируют: «*Starting docker desktop destroys DNS resolution in WSL 2 instances*» (docker/for-win #14023, microsoft/WSL #11445, #4285).

Neon PgBouncer — transaction-режим (`pool_mode=transaction`), до 10 000 клиентских соединений, `default_pool_size = 0.9 × max_connections`; настройки не редактируются. Двойное пуллинг Neon официально НЕ рекомендует.

Scale-to-zero (5 мин на Free) **терминирует существующие idle-соединения** — ваш `ThreadedConnectionPool` обязан переживать это через retry + pre-ping.

Render по умолчанию (сервисы с 2026-02-11) использует Python **3.14.3**; фиксировать нужно через `.python-version`/`PYTHON_VERSION`, не `runtime.txt`. `psycpg2-binary` 2.9.12 имеет колёса cp314.

Cloudflare Pages для Vite/React: preset «React (Vite)», build `npm run build`, output `dist`, root `frontend`; `VITE`-переменные инлайнятся в бандл на билде. [cloudflare](#)

↗

CORS: при Bearer-токене без cookies `allow_credentials` не требуется.

Details

1) DNS-резолуция внешних хостов из контейнера Docker Desktop на Windows+WSL2

Точная причина. Compose-сеть между `postgres`/`backend`/`frontend` работает потому, что Docker поднимает

встроенный DNS-резолвер на `127.0.0.11` в каждой пользовательской bridge-сети — он резолвит имена сервисов через service discovery. Для ВНЕШНИХ хостов этот резолвер форвардит запросы на upstream-DNS Docker-демона, который наследуется от `/etc/resolv.conf` WSL2-дистрибутива. Именно эта цепочка (контейнер → docker DNS → WSL resolv.conf → внешний DNS) рвётся, когда WSL2 генерирует нерабочий `resolv.conf` — особенно после старта Docker Desktop или после сна/возобновления Windows.

Первоисточники:

docker/for-win #14023 («DNS Resolution Fails»): дословно «*Starting docker desktop destroys DNS resolution in WSL 2 instances*». После апдейта Docker Desktop 4.29.0 `ping google.com` из WSL даёт `Temporary failure in name resolution`; помогли даунгрейд до 4.24.0 и правка `bip` в Docker Engine.

microsoft/WSL #11445 («Temporary Failure in Name Resolution With Docker»): репро — открыть WSL, запустить Docker Desktop, `ping google.com` → `Temporary failure in name resolution`. Windows 10.0.19045.4170, WSL 2.1.5.0, Ubuntu 22.04.4, Docker Desktop 4.29.0 с WSL2-backend.

microsoft/WSL #4285 («WSL2 DNS stops working») — точно ваш «после сна» кейс: «*This has happened a few times, and rebooting fixes it. Sometimes just doing `wsl --shutdown` is sufficient to fix it. It correlates with my workstation going to sleep and resuming later with DNS in WSL2 not working*».

microsoft/WSL #9552: наличие некорректного `/etc/wsl.conf` (`generateResolvConf`) может само ломать резолюцию — удаление файла возвращало работоспособность.

Отдельная категория 2025–2026 — **mirrored networking**: `networkingMode=mirrored` в `.wslconfig` в ряде случаев ломает DNS (**microsoft/WSL #40983** — на ARM64 «completely breaks DNS resolution»; **#11002** — при mirrored исчезают все интерфейсы кроме `lo`). Mirrored — это и потенциальное лекарство (снимает VPN/routing-конфликты NAT-режима), и потенциальная причина.

Надёжный, воспроизводимый фикс (по приоритету).

Вариант А (лучший для вашей задачи — прогон миграций). Не гоняйте `psql` из контейнера. Установите `postgresql-client` в WSL2-Ubuntu и запустите:

```
bash
psql "$NEON_DIRECT_URL" -f migration.sql
```

DNS в самом WSL2 обычно работает (в issue #11445 ломается связка *после старта Docker Desktop и внутри контейнера*). Это убирает лишний сетевой слой и совпадает с официальной рекомендацией Neon использовать **direct connection** (без `-pooler`) для миграций.

Вариант В (если нужно именно из контейнера) — явный DNS в Docker Desktop. Settings → Docker Engine:

```
json
{ "dns": [ "8.8.8.8", "1.1.1.1" ] }
```

Apply & Restart. Встроенный резолвер контейнеров начнёт форвардить на публичные DNS, минуя сломанный `resolv.conf`.

Вариант C — починить и закрепить WSL2 `resolv.conf`.

```
bash
sudo tee /etc/wsl.conf >/dev/null <<'CONF'
[network]
generateResolvConf = false
CONF
sudo rm -f /etc/resolv.conf
printf "nameserver 1.1.1.1\nnameserver 8.8.8.8\n" | sudo tee /etc/resolv.conf >/dev/null
sudo chattr +i /etc/resolv.conf # immutable — WSL/Docker не перезапишет
```

Затем из PowerShell: `wsl --shutdown`, перезапустить Docker Desktop.

Вариант D — переключить `networkingMode`. В `%USERPROFILE%\wslconfig`:

```
ini
[wsl2]
networkingMode=mirrored
dnsTunneling=true
autoProxy=true
firewall=true
```

`wsl --shutdown`. Если `mirrored` наоборот ломает (#40983/#11002) — вернитесь на `networkingMode=NAT` с `dnsTunneling=true`.

Чеклист диагностики (сверху вниз):

`ping -c2 1.1.1.1` из контейнера — есть ли IP-связность. Нет → это не DNS, а routing/NAT/VPN.

`cat /etc/resolv.conf` в WSL2 и в контейнере — на какой `nameserver` указывает.

`docker run --rm alpine nslookup google.com` — воспроизводится ли вне вашего `compose`.

`wsl --shutdown` (PowerShell) + рестарт Docker Desktop — снимает «после сна» кейс (#4285).

Корпоративный VPN/split-tunnel — временно отключить и повторить (частая первопричина).

Не помогло → Вариант B (`daemon.json dns`) → C (закрепить `resolv.conf`) → D (`mirrored/NAT toggle`).

2) Neon — связь приложения с PostgreSQL (2026)

(а) Двойное пуллинг (PgBouncer Neon + ваш `ThreadedConnectionPool`). Официальная конфигурация Neon PgBouncer (`docs/connect/connection-pooling`), дословно: `pool_mode=transaction`, `max_client_conn=10000`, `default_pool_size=0.9 * max_connections`, `max_prepared_statements=1000`, `query_wait_timeout=120` — «*These settings are not user-configurable*». Neon прямо предупреждает против двойного пуллинг

(docs/connect/choose-connection): «If you use a pooled Neon connection, avoid adding client-side pooling on top. Let Neon handle it. If you must use client-side pooling, release connections back to the pool promptly to avoid conflicts with PgBouncer». [Neon](#)

Практический вывод: раз у вас уже есть `ThreadedConnectionPool`, непротиворечивы два пути — (1) держать пул приложения **умеренным** и подключаться к pooled-эндпоинту, быстро возвращая соединения; либо (2) подключаться к **direct**-эндпоинту, полагаясь на свой пул. Комбинация «большой app-пул + pooled endpoint» — против рекомендации.

Ограничения transaction-режима (2026), не поддерживаются на pooled-соединении: `SET`/`RESET` (в т.ч. `SET search_path`); `LISTEN`/`NOTIFY`; `WITH HOLD CURSOR`; `PREPARE`/`DEALLOCATE` (SQL-level); временные таблицы `PRESERVE`/`DELETE ROWS`; `LOAD`; session-level advisory locks. Protocol-level prepared statements поддерживаются (PgBouncer ≥1.22, `max_prepared_statements=1000`); в psycopg2 — `cur.execute(query, (...), prepare=True)`.

Для вашей **multi-tenant RLS-схемы это критично**: если tenant-контекст ставится через session-level `SET`/`set_config(...)` — на pooled-эндпоинте он потеряется после завершения транзакции. Решения: (1) `SET LOCAL`/`set_config(..., true)` внутри той же транзакции; (2) direct-эндпоинт; (3) `ALTER ROLE ... SET`. Миграции (RLS, composite FK) — только **direct connection**.

(6) Autosuspend (scale-to-zero) и держащийся пул. По умолчанию Neon suspend'ит compute после 5 минут (300 сек) неактивности. Подтверждено официально (guides/building-resilient-applications-with-postgres), дословно: «Neon automatically suspends idle computes after a period of inactivity (5 minutes by default on the Free plan)... Existing idle connections are terminated when the compute suspends». Запрос по такому «протухшему» соединению из вашего пула вернёт `terminating connection due to administrator command` (SQLSTATE `57P01 admin_shutdown`), либо `08006 connection_failure` / `08003 connection_does_not_exist` (connect/connection-errors). [neon](#)

Cold start: активация compute из idle — официально «typically takes a few hundred milliseconds». (Независимые обзоры называют 300–800 мс / до ~2 сек — это неофициально.) [neon](#)

Как реагировать приложению (официальные рекомендации Neon):

Retry с exponential backoff + jitter: до 5 попыток, задержки от 1 с до 16 с плюс jitter.

Connection timeout 10–15 секунд: дословно «It is thus recommended to set a connection timeout of at least 10–15 seconds». Для psycopg2 — `connect_timeout=10..15`. [neon](#)

Pre-ping / health-check пула: дословно «Configure validation queries (`pool_pre_ping` in SQLAlchemy, validation in HikariCP) to test connections before use» и «Production connection pools detect stale connections and replace them». У «голого» `ThreadedConnectionPool` встроенного pre-ping нет — реализуйте сами: перед выдачей делать `SELECT 1`, при ошибке — выбросить соединение и создать новое. [neon](#)

TCP keepalive: важное уточнение — в официальных доках Neon **НЕ** приводятся рекомендованные значения `keepalives/keepalives_idle/keepalives_interval/keepalives_count` для `libpq/psycopg2`. Советы вида `keepalives_idle=30` в интернете — сторонние. Официальная resilience-стратегия Neon = пул + retry + generous timeout, а не keepalive-тюнинг.

(в) SQL-созданные роли vs `neon_superuser` (2026-нюансы). Роли из SQL (`CREATE ROLE` в `psql/SQL Editor`) получают только базовые `public`-привилегии и **не** входят в `neon_superuser` (в отличие от ролей из Console/CLI/API) — нужны явные `GRANT` на каждый объект и `GRANT CREATE ON SCHEMA public TO <username>` для создания объектов. Лимит: **500 ролей на branch** (в основной справке — «enforced limit of 500»; в исходнике доки формулировка мягче — «recommend keeping it under 500»). Роли принадлежат ветке; при создании child-branch роли родителя дублируются. Известные ограничения даже у владельца: `neondb_owner` не может выдать `pg_create_subscription` (neon issue #9550), не может `ALTER ROLE ... SUPERUSER`. [Neon + 5](#)

Лимиты соединений на Free (0.25 CU): `max_connections=104`, из них **7 зарезервировано** под Neon superuser → **97 доступно приложению** (Neon FAQ: «Seven connections are reserved... a 0.25 CU compute leaves 97 connections for your app»); `default_pool_size≈93` (0.9×104). Free-план 2026 (по сводкам, usage-based после Databricks): 100 CU-hours/мес на проект, 0.5 GB storage на проект, до 100 проектов, 10 branches, autoscale до 2 CU, scale-to-zero — сверяйте с [neon.com/pricing](#).

(г) IP Allow / подключение с Render. IP Allow доступен только на Scale-плане (официально: «Neon's IP Allow feature, available with the Neon Scale plan») — на Free его нет, значит на Free и нет firewall-ограничений, которые надо было бы «открывать» под IP Render; подключение с Render идёт по обычному TLS (`sslmode=require`, `channel_binding=require`). Специализированного гайда «connecting from Render» у Neon **нет**: есть общий [neon.com/docs/guides/render](#), но он под **Node.js/Express + `pg`** («Use Lakebase Postgres with Render»), не под FastAPI/psycopg2, и без firewall-спецправил (только передача `DATABASE_URL` как env). Если позднее включите IP Allow на платном плане — учтите, что у Render на Free нет статических исходящих IP (нужны Render dedicated IPs). [Neon](#)

[neon](#)

3) Render — Python-рантайм и деплой FastAPI (2026)

(а) Дефолтная версия Python. Официальная дока (Setting Your Python Version): **Current default = `3.14.3`**, минимальная — `3.7.3`. Render Changelog дословно: «Newly created Python services now use Python 3.14.3 and uv version 0.10.2 by default». Таблица истории: сервисы с `2026-02-11` и позже → `3.14.3`; `2025-06-12`... `2026-02-11` → `3.13.4`; `2024-12-16`... `2025-06-12` → `3.11.11` и т.д. Ваш лог `Python-3.14.3` — это **актуальный дефолт**, а не аномалия; дефолт «прибит» к дате создания сервиса.

Как фиксировать (по убыванию приоритета, официально):

Env `PYTHON_VERSION` = полная версия (напр. `3.12.9`) — обязательно с patch.

Файл `.python-version` в корне (можно без patch — возьмётся последний patch).

`runtime.txt` в актуальной Render-доке о версиях **не упоминается** (у Heroku прямо «deprecated»).

Правильно в 2026 — `.python-version` или `PYTHON_VERSION`. Отдельного поля `pythonVersion` в `render.yaml` эта

дока не описывает — версия через env/файл.

(б) psycorp2-binary под Python 3.14. Python 3.14.0 вышел **2025-10-07**. `psycorp2` 2.9.12 официально «Add support for Python 3.14»; на PyPI выложены колёса `cp314` (напр. `psycorp2_binary-2.9.12-cp314-...whl`, загружены 2026-04-20). Трекер «Python 3.14 Wheels Readiness» помечает `psycorp2-binary` как публикующий совместимые колёса. **Вывод: на Render с дефолтным 3.14.3 ставьте `psycorp2-binary>=2.9.12` — бинарные колёса есть, компиляция из исходников не нужна.** (Под free-threaded `python3.14t` колёс может ещё не быть, но Render использует обычную GIL-сборку — вас не касается.) [Psycorp](#)

[PyPI](#)

(в) Официальный гайд «FastAPI + Neon на Render». Единого нет. Есть: (1) общий Render-flow (Environment=Python, Build `pip install -r requirements.txt`, Start `uvicorn main:app --host 0.0.0.0 --port $PORT`); (2) Neon-гайд для Render, но под Node. Качественный сторонний разбор именно FastAPI+PostgreSQL на Render — freeCodeCamp. Стек собирается из двух официальных половин.

(г) Ограничения Render Free (2026). Free web service: **512 MB RAM, 0.1 CPU; spin-down после 15 минут** без входящего трафика. Render Docs (Free): «Render spins down a Free web service that goes 15 minutes without receiving any inbound traffic... A Free web service spins back up whenever it next receives an HTTP request or new WebSocket connection. This process takes about one minute». Изменение зафиксировано в changelog (ранее было 30 мин; триггер — входящий HTTP-запрос или WebSocket-сообщение по существующему соединению). Cold start ~30–60 сек. Месячная квота бесплатного компьютера ~750 ч. Free Postgres Render (если бы использовался вместо Neon): 1 GB, **истекает через 30 дней** + 14 дней grace. Health checks и zero-downtime deploy поддерживаются; сервис должен слушать `0.0.0.0` на `$PORT`. Точного официального числа одновременных исходящих соединений **не найдено (не подтверждено)**; практический потолок — 512 MB RAM. Независимый бенчмарк OpenStatus (февр. 2024, Frankfurt): warm TTFB 107–720 мс, p99 до ~16 сек из-за cold start — иллюстративно, не официально. [Livemy + 2](#)

4) Cloudflare Pages — деплой Vite/React SPA (2026)

(а) Подключение монорепо-подпапки. Workers & Pages → Create application → Pages → Connect to Git → установить Cloudflare GitHub App на нужный репозиторий → выбрать репо и production-ветку (обычно `main`). Поля: **Framework preset** («React (Vite)», можно None), **Build command** = `npm run build`, **Build output directory** = `dist`, **Root directory (advanced)** = `frontend`. Официально: «The root directory needs to be specified in cases like monorepos». По таблице пресетов для «React (Vite)»: build `npm run build`, output `dist`. Максимум **5 Pages-проектов на репозиторий**; monorepo-support требует Build System V2+. **Контекст 2025–2026:** Cloudflare сместила фокус на Workers, Pages для новых проектов подаётся как «в режиме поддержки», но для статического SPA полностью функционален. [Stack Harbor + 2](#)

(б) Env-переменные в BUILD для Vite. Vite инлайнит `import.meta.env.VITE_*` на этапе сборки (статическая замена + tree-shaking). Переменные из Pages → Settings → Environment variables попадают в процесс билда как обычные env, и `VITE_`-префиксные корректно оказываются в `import.meta.env` бандла. Cloudflare также инжектит системные: `CI=true`, `CF_PAGES=1`, `CF_PAGES_COMMIT_SHA`, `CF_PAGES_BRANCH`, `CF_PAGES_URL`. **Нюанс/**

предупреждение: это НЕ рантайм-переменные Workers/Pages Functions (`context.env`) — те в статический клиентский бандл не попадают. Всё, что уходит в `VITE_`, становится **публичным** (видно в бандле) — секреты туда нельзя. Значение подставляется на билде → смена переменной требует **пересборки**, а не рестарта.

[cloudflare](#)



(в) SPA-фолбэк в 2026. Cloudflare Pages Docs (configuration/serving-pages) дословно: «*If your project does not include a top-level `404.html` file, Pages assumes that you are deploying a single-page application... Pages' default single-page application behavior matches all incoming paths to the root (/)*». То есть для чистого Vite/React SPA фолбэк работает **из коробки без файла вообще**, если нет `404.html`. `_redirects` (`/* /index.html 200`) поддерживается для гибридных случаев, но в 2025 встречались баги (Community: правило ловило «Infinite loop» и игнорировалось; 308-редиректы). В Workers-подходе (`@cloudflare/vite-plugin`/wrangler) аналог — `assets.not_found_handling: "single-page-application"`. **Рекомендация:** для чистого SPA — полагаться на встроенное поведение (не класть `404.html`); `_redirects /* /index.html 200` держать как явную подстраховку.

[azydeco](#)



(г) Кастомные домены и HTTPS на Free. Free Pages включает кастомные домены и бесплатный SSL. Cloudflare автоматически выпускает/продлевает **Universal SSL** (DV-сертификаты), покрывая apex и first-level поддомены. Для кастомного домена: добавить через Pages → Custom domains (Cloudflare сам создаст CNAME на `<project>.pages.dev`); ручной CNAME без привязки в дашборде даёт **522**. При наличии CAA-записей — разрешить `letsencrypt.org`/`pki.goog`/`ssl.com`. Лимиты Free (по сводкам: unlimited requests/bandwidth, ~500 билдов/мес) — сверяйте с официальной страницей лимитов. [Easton Blog](#)



[Cloudflare](#)



5) CORS-архитектура (Render backend + Cloudflare Pages frontend, Bearer-токен)

Ключевой факт: **при чисто header-based авторизации (Bearer в `Authorization`, без cookies и без `credentials: 'include'`) `allow_credentials` НЕ требуется.** `allow_credentials=True` нужен только для cookies/credentialed-запросов и при этом запрещает wildcard-origin. Раз cookies не используете — `allow_credentials=False`. [Microsoft Learn](#)



Минимальная корректная конфигурация:



python

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://<project>.pages.dev",
        "https://app.example.com", # ваш кастомный домен фронта
    ],
    allow_credentials=False, # cookies не используются → не нужно
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
    allow_headers=["Authorization", "Content-Type"],
    max_age=3600,
)

```

Детали:

Preflight OPTIONS Starlette обрабатывает автоматически — CORSMiddleware перехватывает preflight и отвечает нужными заголовками; отдельный `@app.options`-роут не нужен («*FastAPI middleware handles this automatically*»).

CORSMiddleware добавляйте ПЕРВЫМ (в FastAPI middleware выполняется в обратном порядке добавления): иначе при ошибке в другом middleware ответ уйдёт без CORS-заголовков, и браузер покажет CORS-ошибку вместо реальной («*CORS middleware must be the first middleware you add... because it needs to add CORS headers to all outgoing responses, including errors*»).

`allow_origins` — точный список, **не** wildcard. Для per-branch preview-доменов Cloudflare (`*.pages.dev`) при необходимости используйте `allow_origin_regex`.

Origin — со схемой, **без** завершающего слэша.

`Authorization` явно в `allow_headers`, если не `["*"]`.

6) «Золотой путь» — партнёрские интеграции между провайдерами

Neon ↔ Render: официальный гайд есть (neon.com/docs/guides/render), но под Node.js/Express + `pg`, не FastAPI, и это не deploy-button-интеграция. Render Marketplace-интеграции с Neon как нативной БД **не обнаружено** (Render продвигает собственный managed Postgres).

Neon ↔ Cloudflare: у Neon есть раздел под Cloudflare Pages (neon.com/docs/guides/cloudflare-pages, рядом с Render/Vercel/Netlify в related docs), обычно под Pages Functions/serverless-driver — полного FastAPI-варианта он не покрывает (у вас Pages = только статика).

Render ↔ Cloudflare: Render официально документирует Cloudflare как DNS-провайдера для кастомных доменов (docs.render.com/configure-cloudflare-dns) — это про DNS, не про совместный «SPA+API» reference-деплой.

Единого официального «deploy button»/референс-архитектуры

FastAPI(Render)+Postgres(Neon)+SPA(Cloudflare Pages) от какого-либо из трёх провайдеров **не найдено**

— стек собирается из отдельных официальных гайдов + качественных сторонних (freeCodeCamp).
Наиболее заметная нативная интеграция Neon — **Azure Native Integration** (GA), но она не относится к вашему стеку (Azure-managed orgs, к тому же EOL 31.01.2026).

Recommendations

Этап 0 — разблокировать миграции сейчас (5 минут):

Установите `postgresql-client` в WSL2-Ubuntu и запускайте миграции **из WSL2, не из контейнера**, против **direct** (без `-pooler`) эндпоинта: `psql "$NEON_DIRECT_URL" -f migration.sql`. Чинит DNS-симптом и совпадает с рекомендацией Neon (миграции → direct, т.к. используют `SET`).

Если нужно именно из контейнера — `"dns": ["8.8.8.8", "1.1.1.1"]` в Docker Engine + `wsl --shutdown` + рестарт Docker Desktop.

Порог эскалации: если `ping 1.1.1.1` из контейнера тоже не проходит — проблема не DNS, а routing/VPN; переключайте `networkingMode`, отключайте VPN.

Этап 1 — Neon (соединения):

Приложение → **pooled**-эндпоинт (`-pooler`) с **маленьким** app-пулом и быстрым возвратом соединений; либо direct + свой пул. Миграции и session-state → **direct**.

Multi-tenant RLS: tenant-контекст через `SET LOCAL` / `set_config(..., true)` **внутри транзакции** — иначе на transaction-pooling потеряете контекст. Покройте тестами.

Реализуйте: `connect_timeout=15`, retry до 5 попыток с backoff 1→16 с + jitter, и **самодельный pre-ping** (`SELECT 1`) перед выдачей из пула; при `57P01/08006/08003` — пересоздать соединение). Прямо покрывает scale-to-zero-терминацию.

Cold start на Free неизбежен (~сотни мс официально). Если UX-критично — keep-alive `SELECT 1` каждые ~4 мин (растит CU-hours) либо платный план без scale-to-zero. Триггер апгрейда: первый запрос после простоя стабильно >1-2 сек и это бьёт по пользователям.

Этап 2 — Render:

Зафиксируйте Python: `.python-version` (напр. `3.12` / `3.13`) ИЛИ `PYTHON_VERSION=3.12.9`, чтобы не зависеть от плавающего 3.14.3. Остаётесь на 3.14.3 → `psycpg2-binary>=2.9.12`.

Start: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`; настройте health-check path.

15-мин spin-down + ~1-мин cold start на Free — для стейджинга приемлемо; для демо прогревайте заранее. Порог апгрейда на Starter (\$7): когда cold start мешает демо/webhook'ам.

Этап 3 — Cloudflare Pages:

Root = `frontend`, preset «React (Vite)», build `npm run build`, output `dist`.

URL бэкенда — через `VITE_API_URL` в Pages env; помните: **публично** и требует **пересборки** при изменении; секреты в `VITE_` не класть.

SPA-фолбэк: не кладите `404.html` — встроенное поведение отдаст `index.html`. `_redirects` (`/* /index.html 200`) — опциональная подстраховка.

Кастомный домен — через дашборд Pages (не ручной CNAME); проверьте CAA-записи.

Этап 4 — CORS:

`allow_credentials=False`, точный `allow_origins` (prod-домен + нужные preview), `allow_headers=["Authorization", "Content-Type"]`, `CORSMiddleware` — первым.

Caveats

Неофициальные цифры: cold-start Neon «300–800 мс / до 2 сек» и Render p99 «~16 сек» — из сторонних бенчмарков/обзоров, не official docs. Официально Neon говорит лишь «a few hundred milliseconds».

Free-tier лимиты быстро меняются: цифры Neon Free (100 CU-hours, 0.5 GB, 100 проектов, 10 branches) и Render (512 MB, 15-мин spin-down, 750 ч) частично из вторичных сводок 2026 г.; сверяйтесь с neon.com/pricing и render.com/pricing.

Лимит ролей Neon: «500 per branch» в разных источниках Neon подаётся то как жёсткий лимит, то как рекомендация.

`runtime.txt` **на Render:** явного «deprecated» в Render-доке я не зафиксировал (это у Heroku); Render просто не упоминает его, рекомендуя `.python-version`/`PYTHON_VERSION`. Трактуйте как «не рекомендуется/не документируется».

Render outbound-соединения: официального числа одновременных исходящих соединений не найдено — не подтверждено.

«Золотой путь»: отсутствие единого официального reference-стека — вывод из отсутствия находок, а не прямое отрицание провайдером.

Neon Cloudflare Pages гайд: зафиксирован как related-ссылка neon.com/docs/guides/cloudflare-pages, полный текст не выгружался — детали (serverless driver vs psql) дословно не верифицированы.

TCP keepalive для psycopg2: отсутствие Neon-рекомендаций подтверждено субагентом по официальным документам; если решите добавить keepalive вручную — это будет решение вне официального гайда Neon.